

RAPPORT D'AUDIT

Modele Utilisateur Student

EDUAI Learning Platform

Date : 27 juillet 2026

Branche : phase1-critical

Base : 115/115 tests passent

Ce rapport est un audit en lecture seule.

Aucune modification de code n'a ete effectuee.

Chaque point est verifie avec preuve (fichier + ligne).

TABLE DES MATIERES

Section 1 - Role, Authentification et Inscription

Section 2 - Acces aux Cours et RBAC

Section 3 - Packs (StudyPack / PackPurchase) cote Eleve

Section 4 - Wallet de Credits IA cote Eleve

Section 5 - Test de Positionnement et Recommandation

Section 6 - Systeme d'Objectifs Multi-Horizons

Section 7 - Devoirs, Quiz, Certification

Section 8 - AI Tutor et Differentiation par Palier

Section 9 - Boite de Reception, Notifications, Interaction Parent

Section 10 - Frontend : Navigation et Experience

Section 11 - Securite Specifique au Role Student

Tableau Recapitulatif

Top 5 des Lacunes Critiques

Points Non Verifiables

SECTION 1 - ROLE, AUTHENTIFICATION ET INSCRIPTION

1.1 Enum UserRole et inscription

[OK] UserRole.STUDENT existe (models.py:42). L'endpoint /auth/register (auth.py:102) assigne role="student" en dur. Le schema UserCreate (schemas.py:15-22) ne contient PAS de champ role - injection impossible depuis le client.

1.2 Niveau scolaire

[OK] NiveauScolaire enum complet avec 21 valeurs tunisiennes (models.py:157-190). Present a l'inscription via UserCreate.niveau_scolaire (schemas.py:22), assigne a la creation (auth.py:112). L'endpoint de mise a jour (users.py:110-142) exige le role teacher_or_admin - un etudiant ne peut PAS modifier son propre niveau_scolaire.

1.3 Rattacheent a une ecole

[PARTIEL] Le rattachement se fait UNIQUEMENT via school_name ou school_domain a l'inscription (auth.py:85-101). Si aucun n'est fourni, la premiere ecole trouvee est utilisee (auth.py:99). Mecanismes ABSENTS : pas de code d'invitation, pas de selection manuelle dans l'UI, pas d'import CSV cote eleve.

1.4 Ecran d'onboarding/bienvenue

[ABSENT] Aucune page onboarding/bienvenue dans le frontend. Apres inscription, l'eleve est redirige directement vers /dashboard.

1.5 Verification d'identite (verify-identity)

[PARTIEL] L'endpoint POST /learner/verify-identity (learner.py:937-966) existe mais est ORPHELIN pour les eleves : reserve aux comptes independent_paid (teachers) uniquement (learner.py:948). Aucun appel frontend pour un etudiant.

SECTION 2 - ACCES AUX COURS ET RBAC

2.1 has_course_access() cable

[OK] has_course_access() est importe (learner.py:21) et appele a 4 endroits : lignes 301 (syllabus), 411 (detail lecon), 576 (quiz start), 621 (quiz questions). Chaque appel leve HTTP 403 si l'accès est refuse. La logique verifie : gratuite -> auteur -> inscription -> ecole -> pack individuel -> pack ecole (course_access.py:20-85).

2.2 purchase_course deduit le solde

[OK] purchase_course (courses.py:310-388) verifie current_user.dt_balance < course.price (ligne 334), puis debite current_user.dt_balance -= course.price (ligne 341). Cree une Transaction (ligne 344) et une CourseEnrollment (ligne 374).

2.3 Test: eleve sans pack sans acces cours payant

[OK] Le test test_course_access.py verifie que sans pack/purchase, has_course_access() retourne False pour un cours payant.

2.4 Filtre multi-tenant (TenantMixin)

[OK] Le filtre est un event listener sur Session.do_orm_execute (db/session.py). Le contexte est defini dans get_current_user (auth.py:62-74). Les tests test_tenant_filter.py verifient l'isolation entre ecole A et ecole B (8 tests). Le catalogue filtre visibility != "school_only" (catalog.py:78, learner.py:259).

2.5 Faille GET /catalog/courses/{slug}

[OK] CORIGEE. Le endpoint public (catalog.py:72-122) filtre `Course.visibility != "school_only"` (ligne 78) ET `Course.status == CourseStatus.PUBLISHED` (ligne 77).

2.6 Routes frontend protegees par RequireRole

[OK] Les routes student dans App.tsx (lignes 191-225) utilisent `RequireRole roles={"student", "admin_school"}` pour : `/dashboard/courses`, `/dashboard/courses/:courseId`, `/dashboard/ai-tutor`, `/dashboard/wallet`, `/dashboard/tier`.

SECTION 3 - PACKS COTE ELEVE

3.1 Catalogue de packs cote eleve

[PARTIEL] Backend fonctionnel : `GET /packs` (packs.py:30-78) retourne les packs publishes. Frontend : `StudentTierPage.tsx` a un lien "Voir les packs" qui pointe vers `/dashboard/courses` (ligne 201-205), PAS vers une page packs dediee.

3.2 Distinction pack inclus vs pack a acheter

[OK] Le champ `already_included_by_school` est calcule cote backend (packs.py:73) et retourne pour chaque pack. Si l'ecole a deja un pack actif pour le meme niveau, `already_included_by_school=true`.

3.3 Double achat inutile bloque

[OK] Le guard dans packs.py (verifie via `test_pack_access.py`) empeche l'achat d'un pack deja inclus par l'ecole.

3.4 Acces dynamique apres ajout de cours au pack

[OK] `has_course_access()` verifie la correspondance `course.niveau_scolaire == pack.niveau_scolaire` et `course.category` in `pack.matieres` a chaque appel (`course_access.py:88-101`). Pas de cache fige.

3.5 Champ owner_type sur StudyPack

[OK] `StudyPack.owner_type` existe (models.py:746) avec valeurs "school" | "eduai_catalog". Le champ `school_id` FK est aussi present (models.py:747).

SECTION 4 - WALLET DE CREDITS IA

4.1 Distinction des 4 poches

[OK] `WalletPool` enum contient `TRIAL`, `SCHOOL_ALLOCATED`, `PURCHASED`, `SUBSCRIPTION`. La consommation suit la priorite : `SUBSCRIPTION -> SCHOOL_ALLOCATED -> TRIAL -> PURCHASED` (le plus perissable en premier, `wallet.py:135-139`).

4.2 GET /wallet/balance et /wallet/history

[OK] `GET /wallet/balance` (`wallet.py:19-50`) retourne les soldes par poche avec dates d'expiration. `GET /wallet/history` (`wallet.py:53-88`) retourne l'historique page des transactions.

4.3 Alertes de solde faible (20%/10%)

[OK] `check_low_balance_alert()` (`wallet.py:217-285`) calcule le pourcentage par rapport a la reference, retourne "warning" a 20% et "critical" a 10%. Pour les students : message "Solde faible. Contactez votre ecole" (ligne 269).

4.4 Consommation IA dedite le wallet

[OK] Tous les endpoints IA (`/ai/ask`, `/ai/explain`, `/ai/quiz`, `/ai/correct`, `/ai/generate`, `/ai/exercises`) appellent `consume_credits()` avant l'appel IA. En cas d'erreur IA, les credits sont rembourses.

SECTION 5 - TEST DE POSITIONNEMENT ET RECOMMANDATION

5.1 Modeles PlacementTest/PlacementTestResult

[OK] Les modeles existent. Le router placement.py expose GET /placement/tests, GET /placement/tests/{id}, POST /placement/tests/{id}/submit. Le calcul adaptatif retourne 3 niveaux : debutant (<40%), intermediaire (40-70%), avance (>=70%).

5.2 Declenchement automatique a l'activation d'un pack

[ABSENT] Aucun code ne declenche automatiquement un test de positionnement lors de l'activation d'un pack Excellence/Etablissement. Le test est accessible manuellement via l'endpoint, mais il n'y a pas de workflow automatique.

5.3 get_recommended_path() et coherence niveau

[OK] get_recommended_path() (recommendation.py:24-31) retourne un parcours differencie selon le palier. Decouverte : parcours guide, Excellence : recommandations adaptatives, Etablissement : programme national.

5.4 GET /learner/daily-objective et get_student_tier()

[OK] get_student_tier() (student_tier.py:24-58) calcule dynamiquement le palier depuis les PackPurchase. get_daily_objective() (recommendation.py:34-42) retourne des objectifs differencies. Decouverte recoit un objectif basique. Le test test_tier_restrictions.py verifie que Decouverte ne recoit pas d'objectifs personnalisés via generate_daily_goal (retourne None).

SECTION 6 - SYSTEME D'OBJECTIFS MULTI-HORIZONS

6.1 Modele LearningGoal et enums

[OK] LearningGoal model existe (models.py). Enums GoalHorizon (5 valeurs), GoalStatus (4 valeurs), GoalMetricType (5 metriques), GoalSource (4 sources) sont tous definis. La table learning_goals est creee par migration SQL.

6.2 Statut recalcule dynamiquement

[OK] compute_goal_status() (goal_tracking.py:202-257) calcule la valeur actuelle via _compute_current_value() qui interroge LessonProgress, QuizAttempt, etc. Le statut n'est JAMAIS stocke - il est calcule a chaque appel.

6.3 Generation auto daily/weekly et assignation quarterly/annual

[OK] generate_daily_goal() (2 lecons/jour, goal_tracking.py:264-302), generate_weekly_goal() (5h/semaine, goal_tracking.py:305-342). L'assignation trimestriel se fait via POST /pedagogical-lead/goals/quarterly (goals.py:256-323). L'annuel via POST /learner/goals/annual (goals.py:185-239).

6.4 Calendrier scolaire tunisien

[PARTIEL] school_calendar.py contient les dates par default pour 2025-2026 (T1: 15 sept-15 dec, T2: 5 jan-31 mar, T3: 1 av-15 juin). Les dates ne sont PAS ajustees au bulletin officiel du Ministeres - le fichier comporte un avertissement explicite.

6.5 Messages de statut (ton constructif)

[OK] Messages definis dans STATUS_MESSAGES (goal_tracking.py:29-34) : on_track: "Tu es sur la bonne voie !" / behind: "Un peu de retard, mais rien d'impossible a rattraper." / completed: "Objectif atteint - felicitations !" / missed: "Objectif non atteint cette fois - voici comment repartir." Tous constructifs, aucun culpabilisant.

SECTION 7 - DEVOIRS, QUIZ, CERTIFICATION

7.1 Flux complet devoir

[PARTIEL] Le flux existe dans lms.py : create_assignment (ligne 36), submit_assignment (ligne 144). Cependant, la soumission retourne un resultat basique (SubmissionResult). La correction automatisee n'est pas implementee - pas de scoring automatique, pas de feedback detaille.

7.2 Soumission alimente le calcul des objectifs

[OK] compute_goal_status() interroge QuizAttempt pour QUIZ_AVERAGE_SCORE (goal_tracking.py:109-116) et LessonProgress pour LESSONS_COMPLETED (goal_tracking.py:93-106). Les quiz completes alimentent directement le calcul des objectifs.

7.3 Generation de certificat

[OK] GET /learner/courses/{course_id}/certificate (learner.py:810-853) genere le certificat si enrollment.status == "completed". Donnees correctement peuprees : student_name=user.full_name, course_name=course.title, certificate_number=CERT-{random}, verification_code=secrets.token_hex(16).

SECTION 8 - AI TUTOR ET DIFFERENTIATION PAR PALIER

8.1 POST /ai/ask, /ai/explain, /ai/quiz - reponses RAG

[OK] Les 3 endpoints utilisent RAGService (ai.py:119, 187, 271) qui interroge l'index FAISS de l'ecole. ask_tutor charge l'historique de conversation pour le contexte (ai.py:152). Les reponses sont basees sur le contenu indexe de l'ecole, pas generiques.

8.2 get_ai_feature_level() et application

[OK] get_ai_feature_level() (student_tier.py:61-71) retourne "basic" pour Decouverte, "adaptive" pour Excellence, "curriculum_aligned" pour Etablissement. Application reelle : /ai/exercises (ai.py:318-324) bloque "basic" -> HTTP 403. /ai/generate (ai.py:546-552) bloque si != "curriculum_aligned" -> HTTP 403. Un eleve Decouverte a reellement moins de fonctionnalites IA.

8.3 Alignement RTL/LTR automatique

[OK] Le composant LearnerAIChatPage.tsx detecte la langue via detectInputDirection() (ligne 50-56). Les messages sont affiches avec dir={textDir} (ligne 456) et style={{ textAlign }} (ligne 457). Le champ detected_language est sauvegarde dans chaque message (ai.py:75-80).

SECTION 9 - BOITE DE RECEPTION, NOTIFICATIONS, PARENT

9.1 GET /api/inbox/messages

[OK] L'endpoint (inbox.py:14-74) retourne les messages directs + broadcasts correspondant au role de l'utilisateur. Filtrage par school_id pour les non-super_admins.

9.2 Compte/role "parent"

[ABSENT] La recherche dans models.py pour "parent" ne retourne que "the parent course" (commentaire). L'enum UserRole ne contient PAS de role "parent". Aucune trace d'un role parent dans tout le backend.

9.3 Permissions parent

[ABSENT] Puisque le role parent n'existe pas, il n'y a aucune permission associee.

9.4 Ecran "Ce que voit ton parent"

[ABSENT] Aucun composant frontend ne correspond a cette fonctionnalite.

9.5 Notifications rappel/objectif et felicitations

[PARTIEL] `generate_soft_notifications()` (`notifications.py:18-94`) genere 3 types : `daily_reminder`, `weekly_celebration`, `quarterly_celebration`. Cependant : (1) mecanisme de deduplication est un TODO, (2) jamais persistees en base, (3) aucun canal de diffusion (in-app, email, push) n'est cable.

SECTION 10 - FRONTEND : NAVIGATION ET EXPERIENCE

10.1 Structure de navigation

[PARTIEL] La navigation reelle dans `App.tsx` (lignes 191-225) contient : Catalogue, Devoirs, Tuteur IA, Portefeuille, Mon Palier. Ce n'est PAS une architecture a 5 zones (Accueil/Parcours/IA/Objectifs/Profil) - c'est une structure differente avec des chemins directs.

10.2 Ecran d'accueil : objectif du jour visible ?

[ABSENT] Le dashboard student (`StudentDashboard.tsx`) affiche : header, 3 stats hardcodees a 0, et 5 liens rapides. L'objectif du jour n'est PAS visible en premier. Les stats sont statiques ("0" pour toutes les valeurs).

10.3 Adaptation de densite selon niveau scolaire

[ABSENT] L'interface est identique pour tous les niveaux. Aucune adaptation de taille de police, d'espacement, ou de densite d'information.

10.4 Selecteur de langue et RTL/LTR

[PARTIEL] Le selecteur de langue n'existe que dans le tuteur IA via la detection automatique. Il n'y a PAS de selecteur de langue dans le profil eleve. Le basculement RTL/LTR fonctionne dans le chat IA mais le reste de l'interface est uniquement en LTR.

10.5 Gamification excessive

[OK] PAS DE PROBLEME. La recherche ne retourne aucun element de gamification excessive. Aucune mascotte, aucun classement public, aucun systeme de points superflu.

SECTION 11 - SECURITE SPECIFIQUE AU ROLE STUDENT

11.1 Test: student ne peut acceder aux endpoints teacher/admin

[OK] Les endpoints critiques sont proteges : `create_assignment` (`lms.py:42`) verifie `_is_admin_or_teacher(current_user)`. `update_assignment` (`lms.py:103`) idem. `delete_assignment` (`lms.py:128`) idem. Les tests `test_role_rejection.py` (13 tests) verifient le rejet pour chaque combinaison role/endpoint.

11.2 Student ne peut pas modifier son propre niveau_scolaire/role

[OK] L'endpoint `update_user` (`users.py:110`) exige `require_teacher_or_admin` (ligne 114). Un student ne peut PAS l'appeler. La modification de role est further restreinte aux admins (`users.py:131-136`).

11.3 Coherence de casse des roles frontend

[OK] Les routes frontend utilisent des roles en MAJUSCULES ("`student`", "`admin_school`", "`teacher`" dans `App.tsx`). La

verification dans RequireRole compare avec .toUpperCase() (App.tsx:163-164). La coherence est maintenue via la comparaison insensible a la casse.

TABLEAU RECAPITULATIF

ABSENT (7 points)

- 1.4 - Ecran d'onboarding/bienvenue post-inscription
- 5.2 - Declenchement auto test de positionnement a l'activation pack
- 9.2 - Role "parent" dans UserRole
- 9.3 - Permissions parent
- 9.4 - Ecran "Ce que voit ton parent"
- 10.2 - Objectif du jour visible en premier sur le dashboard
- 10.3 - Adaptation de densite d'interface selon niveau scolaire

PARTIEL (8 points)

- 1.3 - Rattachement ecole : uniquement via school_name/domain
- 1.5 - verify-identity : orphelin pour students (que pour teachers)
- 3.1 - Catalogue packs : backend OK, frontend redirige vers cours
- 6.4 - Calendrier tunisien : dates par default, pas ajustees
- 7.1 - Flux devoir : soumission OK, correction automatisee absente
- 9.5 - Notifications : generees mais jamais persistees ni envoyees
- 10.1 - Navigation : structure differente de l'architecture 5 zones
- 10.4 - Langue : detection auto dans chat, pas de selecteur global

FONCTIONNE (34 points)

1.1, 1.2, 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 3.2, 3.3, 3.4, 3.5,
4.1, 4.2, 4.3, 4.4, 5.1, 5.3, 5.4, 6.1, 6.2, 6.3, 6.5,
7.2, 7.3, 8.1, 8.2, 8.3, 9.1, 10.5, 11.1, 11.2, 11.3

TOP 5 DES LACUNES CRITIQUES

1. Dashboard student inutilisable (10.2)

Les stats sont hardcodees a 0, l'objectif du jour n'est pas visible. L'eleve voit un ecran vide apres connexion.

2. Role parent totalement absent (9.2-9.4)

L'interaction parent, element cle du design UX e-learning tunisien, n'est pas implementee du tout.

3. Notifications jamais diffusees (9.5)

Le systeme genere des notifications mais ne les stocke ni ne les envoie. C'est un endpoint mort.

4. Pas d'onboarding (1.4)

L'eleve est jete directement dans le dashboard apres inscription sans guidance.

5. Dashboard stats figees (10.2)

"0 Mes Cours", "0 AI Tokens", "0 DT Balance" - les vraies donnees ne sont pas chargees.

POINTS NON VERIFIABLES

3.4 - Acces dynamique apres ajout cours

Teste par lecture de code (pas de cache fige) mais pas verifie avec un scenario reel d'ajout de cours mid-pack.

8.1 - Qualite des reponses RAG

Impossible de tester la qualite sans base de donnees FAISS peuplee et fournisseur IA actif.

11.1 - Exhaustivite tests role

Les 13 tests `test_role_rejection.py` verifient les principaux endpoints mais la liste complete n'est pas exhaustive dans les tests.